# Labo 17 — Helm et livraison

*Théorie — comment une stack de onze manifestes devient un paquet versionné, installé, mis à jour et annulé d'une commande ; comment une image part de la CI pour arriver dans le cluster ; et ce qui change quand le cluster n'est plus votre k3s.*

## Objectifs

- Lire un **chart Helm** : `Chart.yaml`, `values.yaml`, `templates/` ; comprendre *release* et *revision*.
- Installer, mettre à jour, annuler : `helm install / upgrade / rollback / history / uninstall`.
- Faire la même chose sans templates avec **Kustomize** (base + overlays) et savoir choisir.
- Adopter une **stratégie de tags et de digests**, et faire tirer k3s depuis un registry.
- Situer la chaîne **CI/CD → cluster**, le **GitOps**, et k3s face à EKS/AKS/GKE/OpenShift.

---

## 1. Du YAML en vrac au paquet

Au labo 16, la stack tenait en une dizaine de manifestes : Deployments, Services, ConfigMaps, Secret, StatefulSet, Ingress, LimitRange. Pour les livrer en recette puis en production, il faudrait les copier et changer le nombre de réplicas, le tag d'image, le profil Spring, l'hôte de l'Ingress… dans des fichiers dupliqués. Et quand la version 1.1 casse, retrouver à la main ce qu'il y avait avant. C'est exactement le problème que `compose.override.yaml` résolvait au labo 09 pour une machine. Kubernetes a deux réponses, complémentaires : **Helm** (des templates et des valeurs) et **Kustomize** (des patchs sur des YAML purs).

> **→ RAPPEL DOCKER**
> `compose.yaml` + `compose.prod.yaml` ↔ `values.yaml` + `values-prod.yaml` ; `podman compose up` ↔ `helm install` ; `podman compose down` ↔ `helm uninstall` (qui, comme `down` sans `-v`, garde les volumes) ; `podman kube play` (labo 11) installait aussi un YAML, mais sans historique ni valeurs. Les tags et digests du labo 10 sont inchangés : ce sont les mêmes images, dans le même registry `localhost:5000`.

## 2. Helm : chart, release, revision

Un **chart** est un dossier (ou une archive `.tgz`) :

```
stack/
├── Chart.yaml          # nom, version du chart, appVersion
├── values.yaml         # toutes les valeurs par défaut — ce qui varie par environnement
├── values-prod.yaml    # surcharges pour un environnement (seulement les différences)
├── templates/          # les manifestes, avec des trous : {{ .Values.api.replicas }}
│   ├── api.yaml  web.yaml  db.yaml  configmap.yaml  secret.yaml  ingress.yaml
│   ├── _helpers.tpl    # morceaux réutilisables (labels communs)
│   └── NOTES.txt       # texte affiché après l'installation
└── files/              # fichiers injectés tels quels : {{ .Files.Get "files/nginx-proxy.conf"
}}
```

Les templates sont du **Go template** : `{{ .Values.api.image.tag }}`, `{{ .Release.Name }}`, `{{ if .Values.web.ingress.enabled }}`, `{{ toYaml .Values.api.resources | nindent 12 }}`. `helm`

`template` rend le YAML sans rien installer ; `helm lint` vérifie le chart. Une **release** est un chart installé sous un nom dans un namespace ; chaque `install`, `upgrade` ou `rollback` crée une **revision**, que Helm stocke dans un Secret `sh.helm.release.v1.<nom>.v<n>` du namespace : c'est ce qui rend `helm history` et `helm rollback` possibles sans serveur.

| Commande | Effet |
|---|---|
| `helm install stack ./stack -n lab17 --create-namespace` | revision 1 ; affiche `NOTES.txt` |
| `helm upgrade stack ./stack --set api.replicas=3 -f values-prod.yaml` | revision suivante ; `--reuse-values` garde les `--set` précédents |
| `helm rollback stack 1` | nouvelle revision « Rollback to 1 » (on ne revient pas en arrière, on avance) |
| `helm history` / `helm get values` / `helm get manifest` | ce qui est installé, avec quelles valeurs |
| `helm uninstall stack` | supprime les objets du chart — **pas** les PVC des `volumeClaimTemplates` |

Un détail qui compte : changer une ConfigMap ne redémarre pas les Pods (labo 14). Le chart annote le template de Pod avec `checksum/config: {{ … | sha256sum }}` ; toute modification de la ConfigMap change l'annotation, donc déclenche un rolling update. C'est l'idiome Helm le plus copié.

**→ HISTOIRE**

Helm naît en 2015 chez Deis comme « apt pour Kubernetes », rejoint la CNCF et devient le standard des *charts* publics (bases de données, Prometheus, ingress controllers…). Helm 2 imposait un composant serveur, Tiller, trop puissant pour les équipes sécurité ; Helm 3 (2019) le supprime : le client parle directement à l'API server avec votre kubeconfig et vos droits RBAC. Helm 4 (fin 2025) garde la même CLI. Kustomize, né chez Google en 2017, est intégré à `kubectl` depuis 2019 (`kubectl apply -k`).

**K3S**

Vous utilisez Helm depuis le labo 11 sans le savoir : k3s installe Traefik par un objet `HelmChart` (CRD) que son contrôleur `klipper-helm` applique ; `helm list -A` montre les releases `traefik` et `traefik-crd` dans `kube-system`. Le binaire `helm` n'est pas fourni : on le télécharge, et il ne connaît pas le kubeconfig de k3s : sans `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml`, il cherche un cluster sur `localhost:8080` et échoue.

## 3. Kustomize : des patchs, pas des templates

Kustomize part d'une **base** (des YAML valides, applicables tels quels) et d'**overlays** par environnement qui la référencent et la modifient : `namespace`, `replicas`, `images` (changer un tag sans toucher au YAML), `labels`, et des `patches` (strategic merge ou JSON patch). Pas de `{{ }}`, pas de valeurs : chaque overlay est lisible seul, et `kubectl kustomize overlays/prod` montre le résultat exact ; `kubectl apply -k overlays/prod` l'applique. Beaucoup d'équipes combinent les deux : `helm template … | kustomize` pour patcher un chart public qu'elles ne veulent pas forker.

| | Helm | Kustomize |
|---|---|---|
| Modèle | templates + valeurs | YAML purs + patchs |
| Distribution | charts versionnés, dépôts, `.tgz` | un dépôt Git |
| Historique / rollback | oui (revisions) | non (c'est Git qui le porte) |
| Convient à | logiciels installés par d'autres (charts publics, produits internes) | vos propres applis, surtout en GitOps |

## 4. Tags, digests et registry

Un **tag** est mobile : `api:1.1` peut désigner demain une autre image, et `latest` est une promesse de surprise. Deux pratiques d'entreprise : des tags **immuables** (le SHA Git ou un SemVer jamais réécrit, posé par la CI), et, pour ce qui tourne en production, le **digest** : `image: localhost:5000/api@sha256:b82020cb…` désigne exactement un manifeste, pour toujours. `podman push --digestfile` donne le digest tel que le registry l'a enregistré — différent de `podman image inspect --format '{{.Digest}}'` sur l'image locale, car le push recompresse les couches ; et `kubectl get pod -o jsonpath='{.status.containerStatuses[0].imageID}'` montre celui que le nœud a réellement tiré.

Jusqu'ici, les images passaient par `podman save | sudo k3s ctr images import`. Le vrai chemin est un **registry** : la CI pousse, le cluster tire. Le `registry:2` du labo 10 suffit : k3s tire `localhost:5000/api:1.1` en HTTP sans configuration (containerd tolère `localhost` en clair). Pour un registry d'entreprise (TLS interne, identifiants, miroir de Docker Hub), k3s lit `/etc/rancher/k3s/registries.yaml` (`mirrors:` et `configs:` avec `auth:` / `tls:`) et il faut `sudo systemctl restart k3s`. Sur tout autre cluster, les identifiants vont dans un Secret `kubernetes.io/dockerconfigjson` référencé par `imagePullSecrets`.

> **→ SÉCURITÉ**
> Un tag mobile est une faille de reproductibilité et une porte d'entrée (qui a le droit de pousser `api:1.1` ?). Le digest ferme la porte ; `imagePullPolicy: Always` avec un tag mobile ne fait que la repousser. Les chaînes sérieuses signent les images (cosign) et refusent au niveau de l'admission (Kyverno, Gatekeeper) ce qui n'est pas signé ou pas scanné. Et le mot de passe du registry n'est jamais dans le chart : `--set` depuis le coffre de la CI, ou un Secret externe (labo 14).

## 5. CI/CD → cluster, et GitOps

La chaîne du labo 10 se prolonge d'une étape :

```
deploy:                                  # extrait de pipeline (GitLab CI, GitHub Actions : même idée)
  script:
    - podman build -t registry.interne/api:$CI_COMMIT_SHA .
    - podman push registry.interne/api:$CI_COMMIT_SHA
    - helm upgrade --install api ./chart -n facturation-prod -f values-prod.yaml
        --set api.image.tag=$CI_COMMIT_SHA --wait --timeout 5m
```

Le *runner* possède un kubeconfig limité à un ServiceAccount (labo 16) ayant `edit` sur **un** namespace ; `--wait` fait échouer le job si le rollout ne converge pas, et un `helm rollback` est la suite du pipeline en cas d'échec. Le **GitOps** inverse le sens : personne n'exécute `helm upgrade` ;

un dépôt Git décrit l'état voulu (charts + values, ou overlays Kustomize), et un agent dans le cluster (**Argo CD**, **Flux**) le lit en continu et réconcilie — la boucle du labo 11, un étage plus haut. La CI ne fait plus que construire, pousser, et **changer un tag dans Git** ; l'historique, la revue et le rollback sont ceux de Git.

## 6. k3s face aux clusters d'entreprise

| | **k3s (ce que vous avez)** | **EKS / AKS / GKE** | **OpenShift** |
|---|---|---|---|
| Control plane | un binaire, chez vous | géré par le cloud, facturé | géré (OpenShift) ou installé |
| `Service: LoadBalancer` | ServiceLB (ports du nœud) | load balancer cloud, IP publique | idem + **Route** (son Ingress maison) |
| StorageClass | `local-path` (disque du nœud) | disques réseau (EBS, Azure Disk, PD) | idem |
| Ingress | Traefik préinstallé | à installer (nginx, ALB…) | Router intégré |
| Identité | kubeconfig = `cluster-admin` | IAM / Entra / Google → RBAC | OAuth intégré, SCC (sécurité stricte, pas de root) |
| CLI | `kubectl` | `kubectl` | `oc` (surensemble) |

Tout le reste — Pod, Deployment, Service, ConfigMap, PVC, RBAC, Helm — est **identique** : c'est l'intérêt d'une API standard. Ce qui change est à la périphérie : qui fournit le load balancer, le disque, l'identité, et combien ça coûte.

## 7. En entreprise

- **Un chart par application**, versionné avec elle ; les `values-<env>.yaml` dans le même dépôt, les secrets ailleurs.
- **Tags immuables posés par la CI**, digests en production, `latest` interdit.
- **Le runner CI a un rôle, pas `cluster-admin`** ; mieux : GitOps, et la CI ne touche plus au cluster.
- **Helm pour ce qu'on installe, Kustomize pour ce qu'on écrit** — ou les deux, sans dogme.
- **k3s est un cluster de développement** fidèle à l'API ; ce que vous y avez appris se transporte tel quel.

---

## À retenir

- Chart = `Chart.yaml` + `values.yaml` + `templates/` ; release = chart installé ; revision = chaque `install/upgrade/rollback`, stockée dans un Secret.
- `helm upgrade … --set` / `-f values-prod.yaml`, `helm history`, `helm rollback 1` (qui crée une revision) ; `uninstall` garde les PVC.
- `checksum/config` sur le template de Pod : une ConfigMap changée redéploie.
- Kustomize = base + overlays (`namespace`, `replicas`, `images`, `patches`), `kubectl apply -k`, aucun template.

- Tag mobile ≠ digest : `--digestfile` au push, `imageID` côté nœud, `image@sha256:` en production.
- k3s tire `localhost:5000` sans config ; un registry d'entreprise passe par `registries.yaml` ou `imagePullSecrets`.
- CI : build → push → `helm upgrade --install --wait` avec un rôle limité ; GitOps : Git décrit, Argo CD/Flux réconcilie.

## Vocabulaire

**chart** : paquet Helm (templates + valeurs). — **release / revision** : chart installé sous un nom / version numérotée de cette installation. — **values** : paramètres du chart, surchargés par `-f` ou `--set`. — **`helm template`** : rendu sans installation. — **Kustomize** : base + overlays, patchs sans templates. — **overlay** : variante par environnement. — **digest** : empreinte immuable d'une image (`@sha256:`). — **registries.yaml** : configuration des registries de k3s. — **imagePullSecrets** : identifiants de registry d'un Pod. — **GitOps** : Git comme source de vérité, réconcilié par un agent (Argo CD, Flux). — **cluster managé** : control plane fourni par un cloud (EKS, AKS, GKE). — **OpenShift** : distribution Kubernetes de Red Hat (Routes, SCC, `oc`).